

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

«Цифровая аналитика»

Отчет о проделанной работе

**«Разработка графического
калькулятора с помощью LLM"»**

Студент,
группы 5130201/40003

_____ Лебедев К. О.

Руководитель,
преподаватель

_____ Мулюха В.А.

«_____» _____ 20__ г.

Санкт-Петербург, 2025

1 Техническое задание

Тема: Графический калькулятор функций.

1.1 Цель

Разработка приложения для построения графиков математических функций одной переменной в декартовой системе координат. Приложение должно предоставлять интуитивно понятный интерфейс с программной клавиатурой для ввода выражений и визуализацией графиков в фиксированном диапазоне.

1.2 Границы проекта

Включено

- Построение графиков функций одной переменной $y = f(x)$.
- Фиксированный диапазон отображения: $x \in [-10^3, 10^3]$, $y \in [-10^3, 10^3]$.
- Одновременное отображение только одной функции.
- Программная клавиатура для ввода математических выражений.
- Поддерживаемые математические операции и функции:
 - Арифметические операции: $+$, $-$, $*$, $/$, $^$ (степень).
 - Тригонометрические функции: $\sin(x)$, $\cos(x)$, $\tan(x)$.
 - Обратные тригонометрические функции: $\arcsin(x)$, $\arccos(x)$, $\arctan(x)$.
 - Гиперболические функции: $\sinh(x)$, $\cosh(x)$, $\tanh(x)$.
 - Логарифмические функции: $\log_{10}(x)$, $\ln(x)$.
 - Степенные функции: $\sqrt{x}$, x^a .
 - Модуль: $|x|$.
 - Экспонента: e^x .
 - Константы: π , e .
- Автоматическая расстановка знака умножения (например, $2x$ интерпретируется как $2 \cdot x$).
- Обработка унарного минуса.
- Проверка корректности введённого выражения.
- Отображение осей координат с подписями.

- Отображение сетки с шагом 10 единиц.
- Отображение текущей позиции курсора в поле ввода.
- Статусная строка с информацией о текущем состоянии.
- Управление с клавиатуры: Enter (построить график), Esc (очистить), Backspace (удалить символ), стрелки (перемещение курсора).

Исключено

- Параметрические функции.
- Функции нескольких переменных.
- Анимация и динамическое изменение графиков.
- Масштабирование и панорамирование (фиксированный масштаб).
- Сохранение графиков в файл.
- Печать графиков.
- Комплексные числа.

1.3 Ограничения

Ограничения на аргументы и значения

- Минимальное значение аргумента: -10^5 .
- Максимальное значение аргумента: 10^5 .
- Минимальное отображаемое значение функции: -10^5 .
- Максимальное отображаемое значение функции: 10^5 .
- Количество точек для построения графика: 2000 (равномерно распределены по диапазону).

Ограничения на ввод

- Максимальная длина выражения: 256 символов.
- Допустимые символы: цифры (0–9), латинские буквы (a–z, A–Z), знаки операций (+, −, *, /, ^), скобки, точка.
- Имена функций: `sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `sinh`, `cosh`, `tanh`, `log`, `ln`, `sqrt`, `abs`, `exp`.

- Константы: π , e . Переменная: x (только строчная).

Ограничения вычислителя

- Собственная реализация парсера математических выражений.
- Стандартный математический приоритет операций.
- Поддержка вложенных функций и скобок.
- Обработка ошибок: деление на ноль, логарифм отрицательного числа, корень из отрицательного числа.

1.4 Функциональные требования

Интерфейс пользователя должен содержать строку ввода с мигающим курсором, программную клавиатуру (цифры, операторы, скобки, константы π , e , переменную x , функции, управляющие кнопки C , $\leftarrow$, $\rightarrow$, $=$, $\Leftarrow$), область построения с осями, сеткой, графиком (синяя линия) и точкой начала координат, а также статусную строку.

Парсер должен выполнять лексический и синтаксический анализ, вычислять значение выражения для заданного x , обрабатывать унарный минус, неявное умножение и константы.

Построение ведётся по 2000 точкам, равномерно распределённым в диапазоне $x \in [-10^5, 10^5]$. Линии через разрывы функции пропускаются.

Взаимодействие. Ввод возможен только через программную клавиатуру (физическая клавиатура блокируется, кроме клавиш Enter/Esc/Backspace/ $\leftarrow$ / $\rightarrow$). При нажатии на кнопку функции автоматически добавляются скобки и курсор помещается внутрь.

1.5 Обработка ошибок

- Пустое выражение — «*Ошибка: Введите функцию*».
- Некорректное выражение — «*Ошибка: Некорректное выражение*».
- Деление на ноль, логарифм / корень из отрицательного числа, переполнение — пропуск соответствующих точек графика.
- Не удалось построить график — «*Ошибка: Не удалось построить график*».

1.6 Технологии реализации

- Язык программирования: C++20.
- Фреймворк: Qt 6 Widgets.
- Сборочная система: CMake.
- Компилятор: Clang (Apple LLVM).
- Графика: QPainter.

1.7 Отклонения от ТЗ

1. Пределы по осям ($x_{\min}$, $x_{\max}$, $y_{\min}$, $y_{\max}$) и шаг сетки сделаны *регулируемыми*, а не фиксированными.
2. Добавлено *масштабирование колесом мыши* и *панорамирование* перетаскиванием левой кнопкой.
3. Количество точек графика *вычисляется автоматически* (пропорционально ширине области построения и ограничено разумным диапазоном) и не задаётся пользователем.

2 История промптов

В этом разделе приведена хронологическая последовательность запросов, переданных языковой модели в ходе разработки. Тексты приводятся в исходной формулировке пользователя.

2.1 Промпт 1 — первичная реализация

«Реализуй приложение в папке `graphicCalculator` в соответствии с техническим заданием. Но — в отличие от ТЗ сделай регулируемые изменяемые пределы и шаг.»

Результат: модель создала структуру проекта (CMake-сборка, точка входа, модули `src/parser/`, `src/widgets/`, `MainWindow`), реализовала собственный парсер математических выражений (лексер, синтаксический анализатор методом рекурсивного спуска, AST, вычислитель), а также все требуемые виджеты: поле ввода с блокировкой физической клавиатуры, экранную клавиатуру, область построения графика и панель настроек с регулируемыми пределами и шагом сетки.

Результат: модель привела последовательность команд CMake (`cmake -B build, cmake -build build -j, open build/GraphicCalculator.app`), указала путь к исполняемому файлу внутри .app-бандла и привела вариант запуска без бандла для отладки.

2.2 Промпт 2 — интерактивное масштабирование

«Добавь масштабирование используя мышь. Также пользователь не должен вводить число точек — оно рассчитывается программой.»

Результат: модель добавила обработчики `wheelEvent`, `mousePressEvent`, `mouseMoveEvent`, `mouseReleaseEvent` в виджет графика. Колесо мыши реализует зум вокруг точки курсора (множитель 1.2 за «деление»), левая кнопка мыши — панорамирование. При наведении показывается всплывающая подсказка с математическими координатами курсора и значением $f(x)$. Сигнал `viewChanged` синхронизирует спинбоксы панели настроек без рекурсии. Поле «Количество точек» удалено из интерфейса; число выборок вычисляется автоматически как $\text{clamp}(2 \cdot W_{\text{plot}}, 500, 8000)$.

Результат: модель подготовила настоящий документ с указанной структурой, используя предоставленные титульный лист и преамбулу.

2.3 Промпт 3 — устранение графических артефактов

«В программе есть баг: при построении сложных графиков возникают графические артефакты — график становится дёрганым либо не отображается полностью (например, $e^{\cos(x)} \cdot \tan(x)$). Это может быть связано с недостаточной плотностью точек либо с алгоритмами оптимизации, которые отображают не весь график. Попробуй разные варианты решения.»

Результат: модель переписала отрисовку кривой, применив сразу три приёма. Во-первых, адаптивная рекурсивная выборка (`refineCurve`) добавляет точки только там, где кривая изгибается или расходится, устраняя изломы у асимптот. Во-вторых, точки вне диапазона Y больше не отбрасываются, а рисуются и обрезаются клиппингом, поэтому ветви доходят до краёв области. В-третьих, определение полюсов переписано: разрыв ставится только при смене знака и вертикальном скачке больше 1.5 высоты экрана, что отличает нечётные полюса (`$\tan$` , `$1/x$`) от крутых непрерывных участков. Дополнительно подавлено наложение подписей осей при мелком шаге сетки.

2.4 Промпт 4 — однократное построение и плавная навигация

«Сделай так, чтобы график строился не динамически при каждом масштабировании, а один раз, после чего масштабированием мы по нему просто передвигаемся. Сейчас проблема — лаги при приближении и удалении; пусть график строится дольше один раз, зато получим плавное отдаление и приближение.»

Результат: модель разделила построение и отрисовку. Метод `rebuildSamples` вычисляет функцию и кэширует точки в математических координатах один раз — при смене функции или явном изменении области, но *не* при жесте мышью. Домен сэмплирования берётся с запасом (6× видимой ширины), поэтому панорамирование и отдаление переиспользуют кэш. Метод `drawCurve` на каждом кадре лишь перепроецирует кэшированные точки в пиксели, благодаря чему зум и панорама стали плавными.

3 Описание использованной языковой модели

3.1 Семейство и версия

В качестве инструмента разработки использовалась языковая модель **Claude Opus 4.7** (идентификатор `claude-opus-4-7`) производства **Anthropic** в режиме интерактивного агента *Claude Code* — официального CLI-клиента Anthropic для работы с исходным кодом.

3.2 Технические характеристики

- Семейство: Claude 4.x (флагманская линейка Anthropic на момент проведения работ).
- Уровень модели: Opus — старшая по возможностям модель в семействе.
- Дата окончания обучающего корпуса: январь 2026 г.
- Поддерживаемая длина контекста — сотни тысяч токенов; в режиме Claude Code контекст автоматически уплотняется при приближении к лимиту.
- Модальность: текст; в режиме Claude Code модель оснащена набором инструментов для работы с файловой системой (`Read`, `Write`, `Edit`), запуска команд оболочки (`Bash`), поиска по проекту, ведения списка задач, а также рядом специализированных агентов.

3.3 Особенности применения

В рамках данной работы модель использовалась как:

1. Архитектор — проектирование разбиения на модули (парсер, виджеты, главное окно, панель настроек).
2. Исполнитель — генерация исходных файлов C++/Qt и конфигурации CMake.
3. Отладчик — запуск сборки, чтение сообщений компилятора и автоматическое исправление ошибок (в частности, конфликта типов `int` и `qsize_t` при сравнении длины строки).
4. Технический писатель — подготовка пользовательской документации и настоящего отчёта.

Все правки в коде вносились через инструменты `Edit/Write`, сборка выполнялась через `cmake -build`, а корректность запуска проверялась запуском бинарного файла в режиме `-platform offscreen`, что подходит для headless-проверки Qt-приложений.

4 Архитектура реализованного приложения

Приложение состоит из четырёх логических слоёв.

1. Парсер выражений (src/parser/):

- **Tokenizer** — лексический анализатор; разбивает строку на токены и автоматически вставляет знак умножения между смежными «открывающим» и «закрывающим» токенами (например, между числом и переменной).
- **Parser** — рекурсивный спуск с грамматикой $expression \rightarrow term \rightarrow unary \rightarrow power \rightarrow primary$; правоассоциативная степень, поддержка вложенных скобок и унарного минуса. Возвращает AST в виде `std::unique_ptr<AstNode>`.
- **Evaluator** — рекурсивный обход AST; при ошибке домена ($\log$ от ≤ 0 , $\sqrt{\cdot}$ от < 0 , деление на ноль, переполнение, нецелая степень от отрицательного основания) возвращает NaN, что трактуется как «пропустить точку».

2. Виджеты (src/widgets/):

- **InputField** — наследник `QLineEdit`; переопределяет `keyPressEvent` и пропускает только Enter, Esc, Backspace и стрелки, эмулируя требование «ввод только через программную клавиатуру».
- **KeyboardWidget** — сетка кнопок, сгруппированных по категориям (функции, цифры, операторы, константы, управляющие клавиши).
- **GraphWidget** — отрисовка через `QPainter`: сетка, оси с подписями, кривая графика; обрабатывает колесо мыши (зум вокруг точки курсора) и перетаскивание (панорама); количество выборок вычисляется по ширине области построения.
- **SettingsPanel** — спинбоксы пределов и шага сетки; синхронизируется с интерактивным масштабированием через сигнал `viewChanged`.

3. Главное окно (src/MainWindow.{h,cpp}) — компоновка, состояние парсера, статусная строка с индикатором позиции курсора, обработка ошибок и сообщения пользователю.

4. Сборка — CMakeLists.txt с qt_add_executable, автоматический moc/rcc/uic, поиск Qt по пути /opt/homebrew/opt/qt на macOS.

5 Заключение и выводы

5.1 Итоги

В ходе работы было разработано полнофункциональное настольное приложение для построения графиков функций одной переменной, удовлетворяющее всем требованиям технического задания. Дополнительно к ТЗ реализованы:

- регулируемые пределы и шаг сетки;
- масштабирование колесом мыши и панорамирование перетаскиванием;
- автоматический подбор количества выборок в зависимости от размеров области построения;
- всплывающая подсказка с координатами курсора и значением функции в точке.

5.2 Достоинства

- **Собственный парсер** реализован «с нуля»: лексер, синтаксический анализатор, AST и вычислитель — без внешних зависимостей кроме Qt. Это удовлетворяет требованию ТЗ и облегчает расширение языка выражений.
- **Корректная обработка ошибок:** сообщения на русском языке, отдельные сообщения для пустого выражения, синтаксической ошибки и пустого графика; разрывы и недопустимые значения вызывают пропуск точек, а не аварийное завершение.
- **Расширенная интерактивность:** зум вокруг точки курсора, панорама, тултип со значениями — решения, естественные для современного UX.
- **Автоматическая адаптация плотности выборки:** избавляет пользователя от выбора числа точек и обеспечивает гладкость при любом размере окна.
- **Чистая модульная архитектура:** один класс — одна ответственность, что облегчает сопровождение.
- **Кроссплатформенность** обеспечивается Qt 6 и CMake.

5.3 Недостатки

- Одновременно отображается только *одна* функция; сравнение нескольких графиков невозможно.
- Шаг сетки не подстраивается автоматически при сильном изменении масштаба — при значительном зуме пользователь должен вручную поправить шаг.
- Отсутствует история введённых выражений и возможность сохранения / загрузки сессии.
- Численная обработка асимптот эвристическая: разрыв определяется по «большому скачку» $|\Delta y|$; для редких функций возможны артефакты в виде разрыва там, где его нет, или короткой соединительной линии.
- Нет аналитических возможностей: производных, нулей функции, точек пересечения, экстремумов.
- Шрифт **Menlo**, использованный в стилях, доступен из коробки только на macOS; на других платформах будет выбран замещающий шрифт.
- Жёсткие границы плоскости ($\pm 10^5$) могут оказаться узкими для отдельных классов задач.

5.4 Выводы

Применение крупной языковой модели Claude Opus 4.7 в режиме агента Claude Code позволило за несколько итеративных запросов довести проект от пустой директории до работающего приложения, удовлетворяющего ТЗ и расширенного интерактивными возможностями. Модель самостоятельно проектировала модули, генерировала исходный код, исправляла ошибки компиляции и адаптировала архитектуру при изменении требований заказчика (переход от фиксированных параметров к регулируемым, а затем — к интерактивному управлению мышью).

Такой подход существенно сокращает время на реализацию типовых desktop-приложений и пригоден как минимум для прототипирования и учебных проектов. Главным ограничением остаётся необходимость содержательной проверки сгенерированного кода человеком — особенно в местах, где автоматический тест не выявляет логических дефектов (например, эвристика разрыва функции).

6 Ссылка на исходный код

Исходный код приложения и настоящий отчёт размещены в репозитории на GitHub:

<https://github.com/Lenebik/GraphicCalculator>

Структура репозитория:

- `graphicCalculator/` — исходный код приложения (C++20, Qt 6 Widgets, CMake);
- `graphicCalculator/src/parser/` — модуль разбора и вычисления математических выражений;
- `graphicCalculator/src/widgets/` — виджеты UI: поле ввода, экранная клавиатура, область графика, панель настроек;
- `graphicCalculator/CMakeLists.txt` — конфигурация сборки;
- `report.pdf` — файл настоящего отчета.
- `prompts.pdf` — файл с запросами к языковой модели.